

Executive Summary

We propose a **cloud-native, microservices-based onboarding and KYC platform** for a mutual-fund company. The system will guide a new investor through registration, identity verification (PAN, Aadhaar), document upload, and approval, culminating in a verified investor dashboard. Our design emphasizes **security and compliance** (encrypting PII, zero-retention), **scalability** (microservices, containerization), and a **frictionless UX** (guided steps, eKYC, liveness checks). The project leverages modern technologies (React, Node.js, MongoDB, AWS Textract/Rekognition, Docker/Kubernetes, Terraform, etc.) and integrates with government services (DigiLocker, UIDAI eKYC). Every step—from “What documents do I need?” to final approval—is **mapped to a technical component or API**, with clear state transitions and robust error handling. In the following sections, we detail the **problem statement, user stories, system architecture, data models, workflow diagrams, tech-stack choices, API examples, security/compliance considerations**, and **DevOps plan**, with pointers to authoritative sources at each point.

Problem Statement & Goals

Mutual-fund companies are **mandated by law (e.g. PMLA 2002, SEBI guidelines)** to verify every investor’s identity and address before allowing transactions [35†L18-L22] [35†L48-L51] . KYC (Know Your Customer) prevents fraud, money laundering, and unauthorized access [35†L18-L22] [35†L48-L51] . However, traditional KYC is paperwork-heavy and cumbersome for users. In the digital age, **conversion (dropout) rates are very high**: studies report ~60–70% of users abandon long, unclear onboarding processes [22†L1218-L1227] . Our goal is to build a **secure, compliant KYC flow that feels quick and clear to an 8th-grade user**. This means:

- **Compliance**: Enforce Aadhaar/PAN verification per UIDAI/SEBI rules (store Aadhaar only in a secure vault, no raw PII in logs [8†L82-L90] [35†L18-L22]).
- **Security**: Use encrypted communications, JWT auth, and third-party verifications (e.g. UIDAI, DigiLocker) to avoid storing sensitive data (a “zero-retention” architecture [14†L931-L939]).
- **User-Centric Design**: Show a pre-checklist of required documents and time estimate [22†L1244-L1253] ; guide the user through each step (OTP verification, PAN/Aadhaar, selfie check), with clear instructions to minimize confusion [22†L1244-L1253] [35†L98-L102] .
- **Automation & Speed**: Automate document reading using OCR/ML (e.g. AWS Textract) and face matching (e.g. AWS Rekognition) to complete most checks instantly [40†L84-L92] [38†L9-L16] .
- **Scalability**: Use microservices and cloud infrastructure (container orchestration, auto-scaling) so the system can handle many concurrent users.

By meeting these goals, **the user will experience a step-by-step digital onboarding that feels fast and safe**, while we meet all regulatory requirements behind the scenes.

User Stories (Beginner's Perspective)

1. Landing & Ready Checklist

2. *Scenario*: "Rita, a new investor, opens the app."

3. *Experience*: Instead of a blank login page, Rita sees a **"Before You Start" screen** listing what she needs: PAN card, Aadhaar, mobile (linked to Aadhaar), photo ID or bank cheque, and an estimated time ("~5 minutes"). This follows best practice of showing all requirements upfront [22†L1244-L1253] . She clicks **"I'm Ready"** and is reassured by a progress bar.

4. Account Creation

5. *Scenario*: "Rita decides to register."

6. *Experience*: She enters **Email, Mobile Number, and Password**. The system sends an OTP to her phone (via SMS). She enters the OTP to verify her mobile. On success, the backend creates a user record (storing a hashed password with bcrypt, not plaintext). A JWT token is issued for her session. *(Tech: Frontend (React) calls backend API; Backend (Node.js/Express) checks if email exists, hashes password, saves to MongoDB, returns JWT [40†L84-L92] .)*

7. Profile Completion

8. *Scenario*: "Rita is logged in and now fills profile."

9. *Experience*: The app presents a form for *Name, DOB, Gender, Occupation, Income, Address, Bank Account (with IFSC), Nominee details*, etc. The fields are clearly labeled. When she saves, the data is validated and stored in the user's profile document. *(Tech: React form → API call → Backend/ UserService → MongoDB "profiles" collection.)*

10. KYC Method Selection

11. *Scenario*: "Rita must verify her identity."

12. *Experience*: She sees a choice: **"Fetch via DigiLocker (Recommended)"** or **"Upload Documents Manually"**. The DigiLocker option is explained ("Secure gov't portal for your Aadhaar/PAN"). This aligns with recommendations to use eKYC whenever possible. If she chooses DigiLocker, the app redirects her to DigiLocker (OAuth flow). If manual, the next steps show file pickers for PAN/Aadhaar upload.

13. PAN Verification

14. *Scenario A: DigiLocker chosen* – Rita is prompted to authorize the app on DigiLocker. After login (using her Aadhaar OTP), DigiLocker returns her PAN card data (digitally signed PDF). The backend receives

this, stores the PDF in AWS S3, and immediately fills her PAN details. The app confirms: "PAN verified" with name and number.

15. *Scenario B: Manual upload* – Rita uploads a photo of her PAN card. The backend saves it to S3 and runs OCR (AWS Textract) to extract the PAN number and name. The app then shows the extracted text and asks her to confirm or correct it. (*Tech: AWS Textract API to parse image; the confidence text is sent back for UI.*) If the user-entered PAN/Name does not match the extracted, it flags an error and asks for re-check. In either case, the backend can optionally call a PAN validation API (e.g. NSDL/KARZA) to ensure the PAN is valid.

16. Aadhaar Verification

17. *Scenario A: DigiLocker* – Similarly, DigiLocker provides the Aadhaar details (masked), and we auto-fill her name and address.
18. *Scenario B: Manual* – Rita uploads her Aadhaar card or enters her Aadhaar number. The system sends an OTP to the mobile registered with Aadhaar. She enters the OTP; the backend calls the UIDAI Aadhaar Authentication API (via an AUA/KUA) to fetch her eKYC XML. The app shows the retrieved name/address.
19. *Tech Note:* We do **not store her plain Aadhaar number** in our database (by law, only a hashed reference/token can be kept [8†L82-L90]). We only store a UIDAI-issued pseudonym (`uid-token`) in logs if needed [8†L179-L187] .

20. Photo (Selfie) & Liveness Check

21. *Scenario:* "The app asks for a selfie."
22. *Experience:* Rita uses her webcam or phone camera to take a selfie. The system runs a **liveness detection** check (e.g. asks her to blink or smile) to ensure the image is of a live person. It then compares her face to the photo on her PAN/Aadhaar (if allowed) via a face-match API. (*Tech: AWS Rekognition Face Match / Face Liveness APIs [38†L9-L16]* .) If match is high, the app shows "Face verified – it's you!". If not, she may retry or the case goes to manual review.

23. Review and Submit

24. *Scenario:* "Rita reviews all info."
25. *Experience:* Before final submission, the app presents a summary: entered details, extracted PAN/Aadhaar info, and her selfie check status. She confirms everything and clicks "Submit for Verification". The backend marks her KYC request as "**Pending Review**".

26. User Dashboard

27. *Scenario:* "Rita waits for approval."

28. *Experience*: Rita now sees her **Investor Dashboard**. It shows her KYC status (“Pending”, “Approved”, or “Action Required”), with timelines. It also shows her profile info, uploaded documents, and (once approved) a “Start Investing” button. As KYC progresses, she receives notifications (email/SMS) for key events (e.g. “Your PAN is verified”, “Your Aadhaar OTP is confirmed”, “KYC approved” etc.).

29. Error/Resubmission Flow

- *Scenario*: “Something went wrong.”
- *Experience*: If any check fails (e.g. PAN OCR mismatch, liveness fail, or admin finds a blurry image), Rita gets a clear message: e.g. “PAN photo not clear. Please upload again.” She can re-upload the specific document or retake the selfie. The workflow then re-enters that step. All errors are explained in plain language (no technical jargon).

This **end-to-end user flow** ensures a novice can intuitively complete KYC with minimal confusion. At each step we explain *why* (e.g. “Aadhaar helps verify address securely”) and give *time estimates* and *progress indicators*. We adopt best practices: clear checklists, progressive disclosure, and fallback options (like alternate documents) [22†L1244-L1253] .

High-Level System Architecture

```
flowchart LR
    subgraph Client
        A[User Device (Web/Mobile App)]
    end
    subgraph Frontend
        B[React (Web) / React Native (Mobile)]
    end
    subgraph API_Layer
        C[API Gateway / Load Balancer]
    end
    subgraph Services
        D[Auth Service]
        E[User/Profile Service]
        F[KYC Service]
        G[Document Service]
        H[Admin Service]
        I[Notification Service]
        J[OCR/AI Service]
    end
    subgraph DataStore
        K[(Users DB)]
        L[(KYC DB)]
        M[(AuditLog DB)]
        N[(Notification DB)]
    end
    subgraph External
```

```

O[DigiLocker/Aadhaar (API)]
P[AWS S3]
Q[AWS Textract]
R[AWS Rekognition]
S[Email/SMS Gateway (SES/SNS)]
T[CI/CD Tools]
U[Monitoring/Logging]
end

%% User flow arrows
A --> B --> C
C --> D
C --> E
C --> F
C --> H
C --> I
D --> K
E --> K
F --> L
H --> L
F --> G --> P
G --> Q
F --> R
I --> S
F --> M
A -- DigiLocker Flow --> O
O --> F
T --> Services
Services --> U

```

Figure: System architecture. The **frontend** (React-based web or mobile) talks to an **API gateway** (Node.js/Express) which routes requests to microservices:

- **Auth Service** (handles registration/login, JWT issuance) using a **Users DB** (MongoDB).
 - **User/Profile Service** (saves extended profile data).
 - **KYC Service** (orchestrates the KYC flow, tracks status).
 - **Document Service** (receives file uploads, stores in S3).
 - **OCR/AI Service** (runs Textract/Google Vision and Rekognition for OCR and face checks).
 - **Admin Service** (dashboard for compliance officers to review KYC requests).
 - **Notification Service** (dispatches emails/SMS via SES/SNS).
- Each service has its own database (NoSQL or SQL) for isolation. We log all actions to an **AuditLog DB** for compliance. External integrations include DigiLocker/Aadhaar APIs, AWS S3 (for files), AWS Textract (OCR) and Rekognition (face/liveness), and email/SMS gateways. DevOps tools (shown) handle CI/CD into this containerized cluster, and Monitoring/Logging captures metrics and logs.

Microservices & Component Responsibilities

- **Frontend (React/React Native):** Responsive UI with form wizards and progress UI. Manages user auth state with JWT.
- **Auth Service:** Endpoints `/register`, `/login`, `/refresh-token`. Verifies credentials, hashes passwords (bcrypt), issues JWTs. Keeps login attempts logs. Tech: Node.js with Express, using Passport or JWT libraries. Uses **MongoDB** (users collection with indexes on email/phone) or alternatively a relational store.
- **User/Profile Service:** Endpoints `/profile` (GET/PUT). Stores extended info (name, address, bank details, nominee). Tech: Node.js/Express (or a GraphQL API) with MongoDB or PostgreSQL.
- **KYC Service:** Coordinates KYC steps. Exposes endpoints like `/kyc/start`, `/kyc/status`, `/kyc/resubmit`. Manages the **KYCRequests** collection: `{ kycId, userId, status, submittedAt, updatedAt }`. Handles transitions (Pending→UnderReview→Approved/Rejected). Enforces state machine (see below).
- **Document Service:** Endpoint `/kyc/upload`. Receives file uploads (PAN/Aadhaar images), validates file types/size, streams to **AWS S3** with encryption. Saves metadata in a **Documents** collection: `{ docId, kycId, type, s3Path, uploadTime, ocrData, ocrStatus }`. Triggers OCR jobs.
- **OCR/AI Service:** Consumes new uploads (e.g. from a queue or event). Calls AWS Textract (via SDK) to analyze images. Parses JSON response to extract fields (PAN number, name, etc.) and updates the Document record. Calls AWS Rekognition for face comparison/liveness. This can be a separate microservice or integrated into KYC Service.
- **Admin Service:** Provides a protected admin UI (React/Bootstrap). Admins can query KYC requests (search by user, PAN), view submitted documents (download from S3), extracted data, and audit trail. Endpoints: `/admin/kyc/:id/approve` or `/reject` with comments. On action, it updates KYC status and writes to audit log.
- **Notification Service:** Sends email/SMS for events (registration, OTP, KYC status updates). Uses AWS SES (email) and AWS SNS or Twilio (SMS). Has a queue or topic subscriptions. Ensures idempotent delivery.
- **Audit Logging:** Every important event (login, upload, admin decision) is recorded as an audit entry: `{ timestamp, userId/adminId, action, details }`. Stored in **AuditLog DB** or append-only store (maybe Elasticsearch for quick search). We do not log sensitive PII (UIDAI forbids logging Aadhaar or OTP `[8†L82-L90]`).

Each service is containerized (Docker) and independently deployable. They communicate via REST/JSON or internal messaging (AWS SQS or Kafka) for async tasks (e.g. after upload, trigger OCR).

Data Model

```
erDiagram
    USERS {
        string _id PK
        string email
        string phone
        string passwordHash
        string role
    }
```

```

}
PROFILES {
    string userId FK
    string name
    date dob
    string gender
    string occupation
    number income
    string address
    string ifsc
    string bankAccount
    string nominee
}
KYC_REQUESTS {
    string kycId PK
    string userId FK
    string status          "Enum: Pending, UnderReview, Approved, Rejected"
    date createdAt
    date updatedAt
}
DOCUMENTS {
    string docId PK
    string kycId FK
    string docType          "PAN, Aadhaar, Photo, etc."
    string s3Url
    string ocrText
    string ocrStatus        "Pending, Done, Failed"
    date uploadedAt
}
AUDIT_LOG {
    string logId PK
    string userId
    string actorId          "Who performed action (user or admin)"
    string action            "LOGIN, UPLOAD_PAN, APPROVE_KYC, etc."
    string details          "JSON or text"
    datetime timestamp
}
USERS ||--|| PROFILES : has
USERS ||--|{ KYC_REQUESTS : initiates
KYC_REQUESTS ||--|{ DOCUMENTS : includes
USERS ||--|{ AUDIT_LOG : \"owns\"

```

- **Users Collection:** Index on `email` (unique) and `phone` (unique) for quick login lookup.
- **KYC_Requests:** Each has a `userId` index. We also index by `status` to efficiently find all pending KYC.
- **Documents:** Index by `kycId`. `ocrText` can be stored or avoided (can regenerate if needed).

- **Audit_Log:** Might use a time-series or append-only DB (e.g. MongoDB capped collection). TTL index could purge old logs beyond retention.

This schema allows tracking each step clearly: a user → profile → many KYC requests (if resubmitted) → each request has multiple documents (PAN, Aadhaar, selfie, bank proof).

Main API Endpoints (sample JSON)

Below are examples of key REST endpoints. All responses use HTTP status codes for success (200/201) or error (400/401/500), and carry a JSON payload.

Registration

```
POST /api/register
Request 201:
{
  "name": "Rita Singh",
  "email": "rita@example.com",
  "phone": "+919876543210",
  "password": "SecureP@ss1"
}
Response 201 (created):
{
  "message": "User created. Verify OTP sent to mobile.",
  "userId": "abc123"
}
Response 400:
{ "error": "Email already in use." }
```

Login (JWT)

```
POST /api/login
Request 200:
{
  "email": "rita@example.com",
  "password": "SecureP@ss1"
}
Response 200:
{
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "expiresIn": 3600
}
```

```
Response 401:
{ "error": "Invalid credentials." }
```

Get User Profile (protected)

```
GET /api/profile
Headers: Authorization: Bearer <token>
Response 200:
{
  "userId": "abc123",
  "name": "Rita Singh",
  "dob": "2001-08-12",
  "address": "123 Main Street",
  "ifsc": "ABCD0123456",
  "bankAccount": "1234567890",
  "nominee": "Rahul Singh"
}
```

PAN/Aadhaar Verification (Sample)

```
POST /api/kyc/verifyPAN
Headers: Bearer token
Request 200:
{ "panNumber": "ABCDE1234F", "fullName": "Rita Singh" }
Response 200:
{ "status": "verified", "source": "DigiLocker" }
```

or

```
POST /api/kyc/uploadDocument
Headers: Bearer token
FormData: { file: <image>, docType: "PAN" }
Response 201:
{ "docId": "doc789", "ocrStatus": "pending" }
```

Then OCR service updates later:

```
PUT /api/kyc/document/doc789/ocrResult
{
  "ocrText": {
    "name": "RITA SINGH",
    "pan": "ABCDE1234F",
    "dob": "12/08/2001"
  }
}
```

```
  },  
  "ocrStatus": "done"  
}
```

Selfie/Liveness

```
POST /api/kyc/verifyFace  
Headers: Bearer token  
FormData: { file: <selfie_image> }  
Response 200:  
{ "matchScore": 0.98, "liveness": true }
```

Submit KYC Request

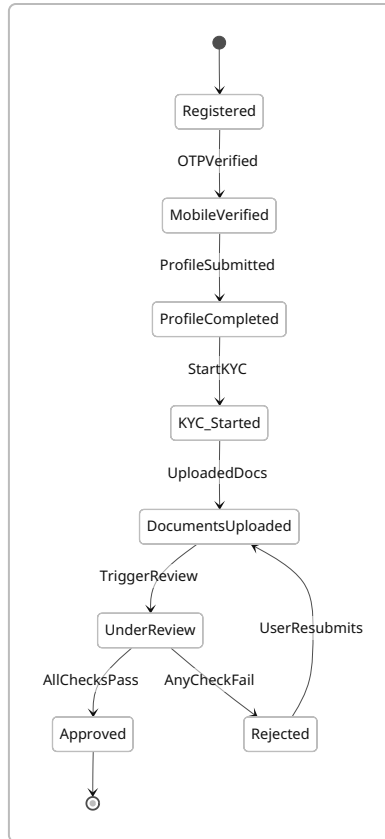
```
POST /api/kyc/submit  
Headers: Bearer token  
Response 200:  
{ "message": "KYC request submitted", "kycId": "kyc456", "status":  
  "UnderReview" }
```

Admin Approve/Reject (protected admin role)

```
POST /api/admin/kyc/kyc456/approve  
Headers: Bearer <admin-token>  
Response 200:  
{ "status": "Approved", "approvedAt": "2026-06-22T14:05:00Z" }
```

Each endpoint uses clear HTTP codes: 201 for created resources, 200 for OK, 401 for unauthorized, 400 for client errors, 500 for server errors. Input validation returns 400. Errors never expose sensitive info. JWT middleware rejects expired/invalid tokens (401).

Document Verification Workflow (State Machine)



Each **KYC request** goes through states:

- **Registered**: initial (user has account).
- **MobileVerified**: after phone OTP.
- **ProfileCompleted**: after user fills profile.
- **KYC_Started**: user initiates KYC.
- **DocumentsUploaded**: all required docs (PAN/Aadhaar) are uploaded or fetched.
- **UnderReview**: backend/Admin is verifying them.
- **Approved/Rejected**: final outcome.

On rejection, the user may resubmit documents, which cycles back to **Uploaded**. All transitions are accompanied by status updates in the UI and notifications. This ensures users and admins see exactly what's happening at each step. Error handling (e.g. OCR failure) triggers a **Resubmit** path.

Technical Stack and Rationale

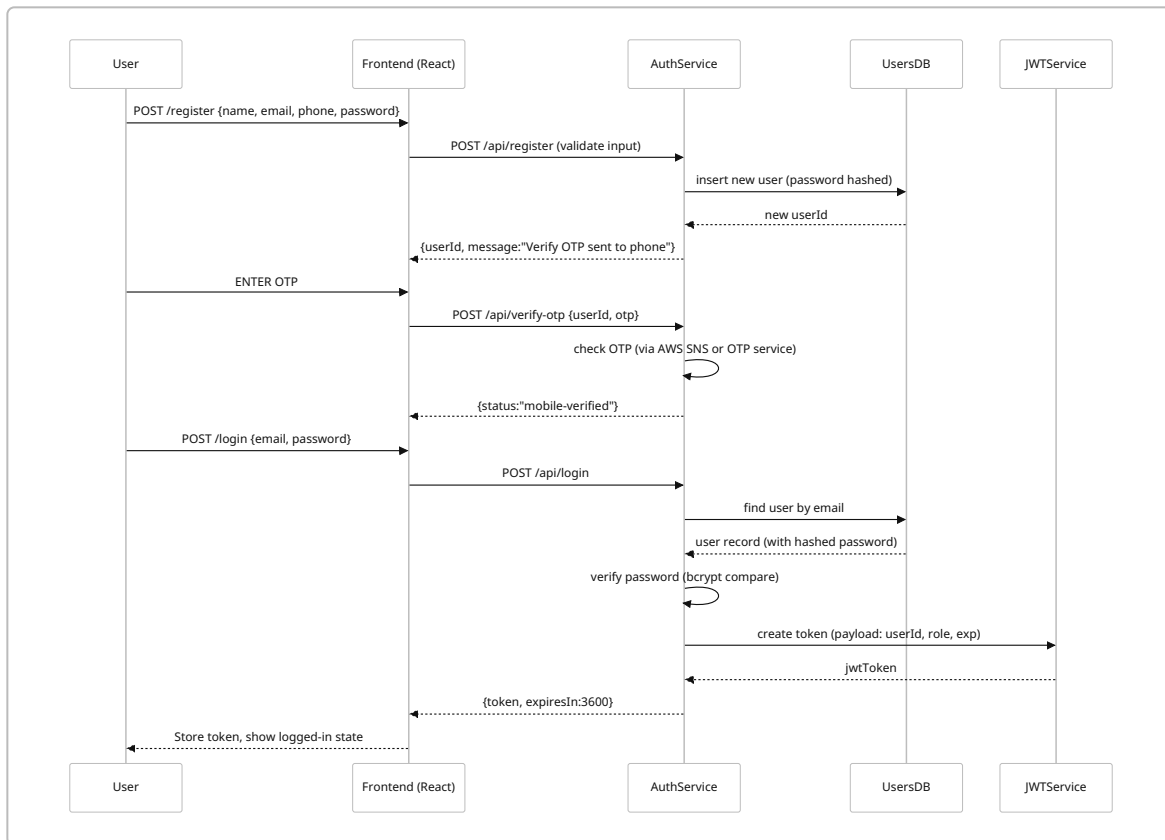
We choose proven fintech technologies for each part:

- **Frontend:** *React.js* (web) and *React Native* (mobile) for a responsive UI [11†L239-L244] . These support fast development of forms and OAuth flows (e.g. DigiLocker).
- **Backend (Microservices):** *Node.js with Express/Koa* (or equivalently Java Spring Boot) for REST APIs. Microservices (Auth, User, KYC, Document, Admin, Notification, Audit) are independently deployable. This pattern is used by Groww and others for scalability [11†L239-L244] .
- **Authentication:** *JWT tokens* (signed with a secure secret from AWS Secrets Manager) for stateless auth [14†L972-L980] . We use *bcrypt* to hash passwords. OAuth is used for DigiLocker.
- **Database:** *MongoDB* (document store) for flexibility in storing varied KYC data and audit logs. We store user profiles and KYC requests in separate collections, keyed by `userId`. (Alternatively, *PostgreSQL* or *AWS DynamoDB* could be used; Mongo is a common choice in modern fintech stacks.) All sensitive fields (like Aadhaar) are encrypted and/or not stored, per UIDAI rules [8†L82-L90] . Key indexes: unique on `email/phone`, and indexes on `userId`, `status` fields for fast queries.
- **File Storage:** *AWS S3* to store all uploaded images and generated PDFs. Use server-side encryption (SSE-S3 or SSE-KMS) and signed URLs. We store only references (URLs) in MongoDB.
- **OCR/Document Analysis:** *AWS Textract* or *Google Cloud Vision* for reading IDs. Textract can extract structured fields from forms/tables [40†L84-L92] . The service calls Textract asynchronously (using the AWS SDK), parses the JSON into our document model (see sample JSON blocks above).
- **Face Match & Liveness:** *AWS Rekognition* provides both face comparison and a Face Liveness API [38†L9-L16] . We use Rekognition to compare the user's selfie against the ID photo, and optionally require a "liveness" video to prevent spoofing. (Rekognition Face Liveness checks a short selfie video to ensure a real person is present [38†L9-L16] .) This yields a confidence score and boolean liveness result.
- **Messaging:** *AWS SNS/SQS* for internal queues, and *AWS SES* (or Twilio) for notifications. For example, after a KYC status change, the KYC Service publishes an event; the Notification Service consumes it and sends an email/SMS. SNS topics or SQS queues decouple services.
- **Logging & Audit:** *Elasticsearch/Kibana* or *AWS OpenSearch* to aggregate logs and audit trails. Each service logs structured JSON (timestamp, service, action, `userId`). *CloudWatch* (if on AWS) can ingest Docker logs. Strict logging rules forbid any PII (Aadhaar, OTPs) [8†L82-L90] .
- **Monitoring:** *Prometheus* (metrics) and *Grafana* (dashboards) for custom service metrics (requests/sec, error rates, KYC throughput). *AWS CloudWatch* for infrastructure metrics. Alerts on failures/ timeouts (e.g. OTP API failing).
- **Containerization & Orchestration:** *Docker* images for each service, deployed to *Kubernetes (EKS)* or *AWS ECS/EKS*. We use *Terraform* or *AWS CloudFormation* for IaC (defining VPC, subnets, DB instances, EKS cluster, etc.).
- **CI/CD Pipeline:** *GitHub Actions* or *Jenkins* automates build, test, security scans, and deployment. On each push, run unit tests, build Docker images, push to ECR, and deploy to a staging namespace. Merges to main trigger production deploys (with manual approval).
- **Secrets & Encryption:** Use *AWS Secrets Manager* or *HashiCorp Vault* to store DB passwords, JWT secrets, DigiLocker API keys, etc. All HTTP traffic is HTTPS/TLS. Databases use encryption-at-rest (MongoDB uses encrypted volumes, or AWS has KMS encryption). We ensure CSP/helmet in the frontend for security.

This stack is inspired by leading fintechs (Groww uses React, Spring Boot and microservices [11†L239-L244] ; others use AWS for scalability).

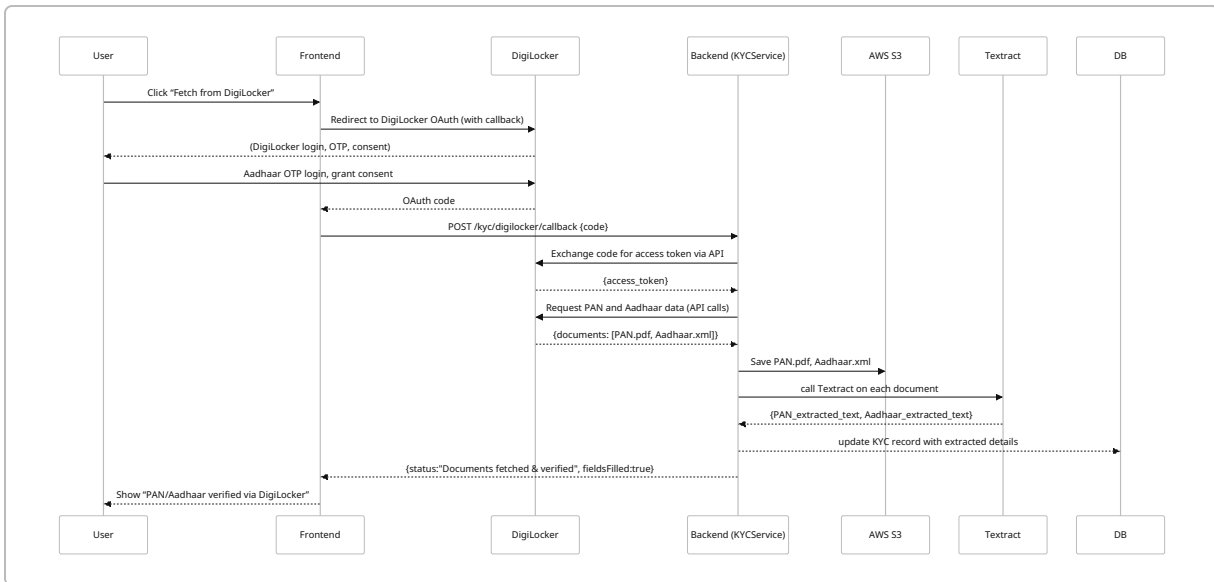
Sequence Diagrams

1. Registration & Login (JWT)



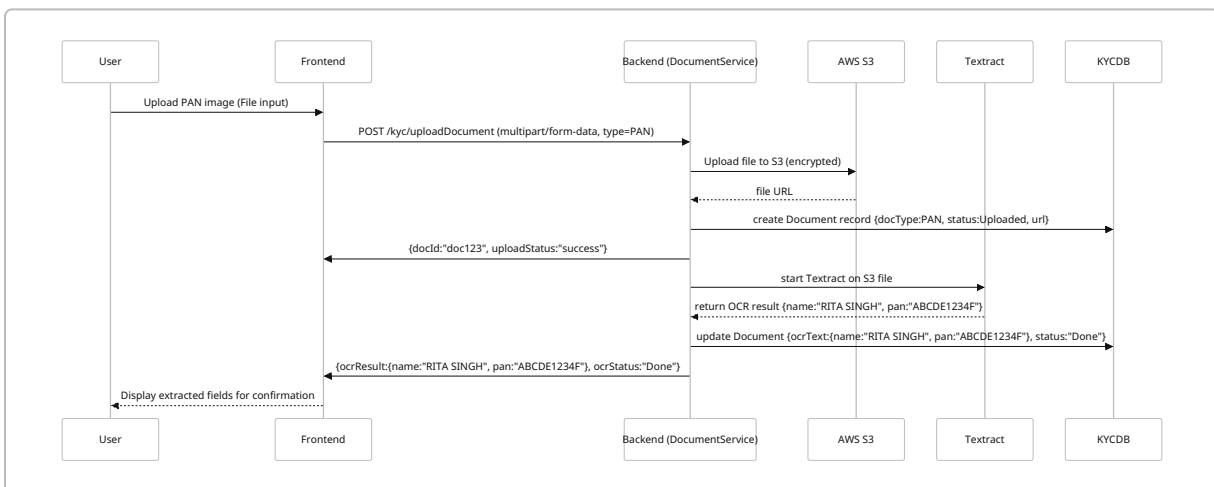
This shows the **JWT login flow**: user registers, verifies OTP, logs in, and receives a JWT for further requests. Passwords are never stored or logged in plaintext. Login failures (wrong password) return 401 and are logged.

2. DigiLocker eKYC Flow



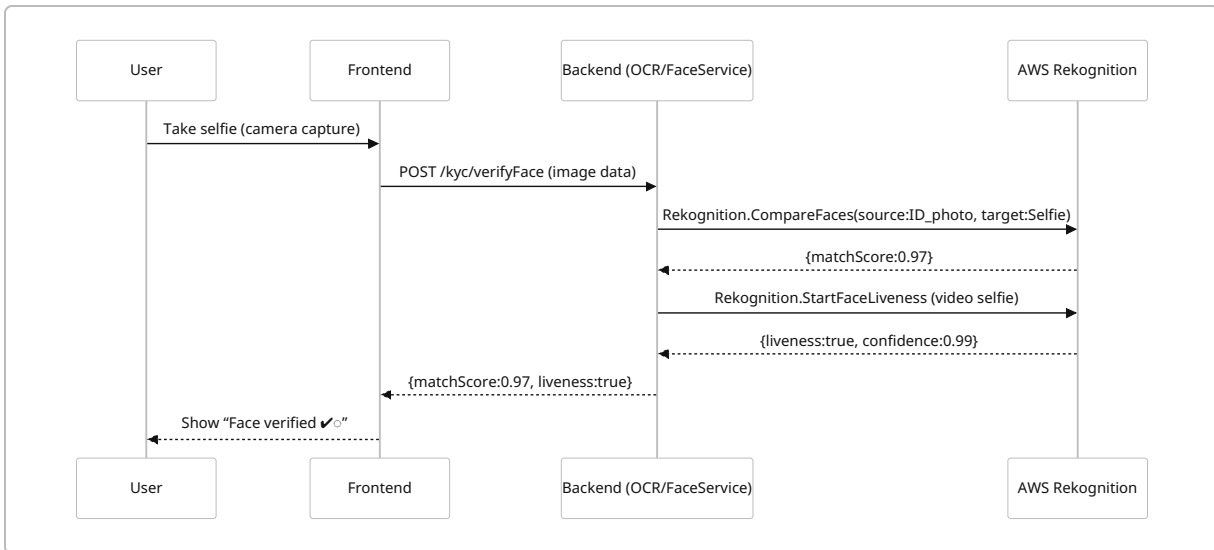
This sequence shows using **DigiLocker APIs**: after OAuth, the backend fetches signed documents from DigiLocker, stores them, and runs OCR to fill user data automatically. This avoids manual upload and OCR errors.

3. Manual Upload + OCR



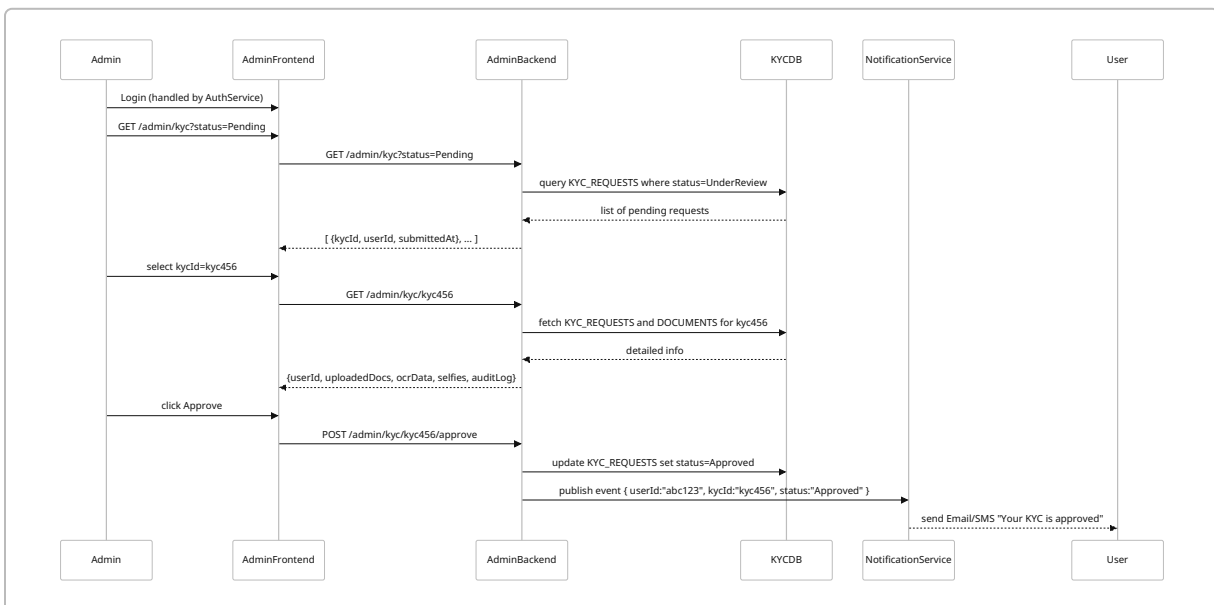
After upload, the backend stores the image in S3 and immediately calls AWS Textract. Once OCR completes, the extracted text is sent back to the frontend for the user to review before final submission. Similar flow happens for Aadhaar upload.

4. Face Match & Liveness



This shows a call to AWS Rekognition: comparing the live selfie against the ID photo (if available), plus a liveness video check. In practice, the selfie capture might be the same video. The response gives a confidence score and liveness boolean. If liveness is false (spoof), the backend rejects the KYC step.

5. Admin Review



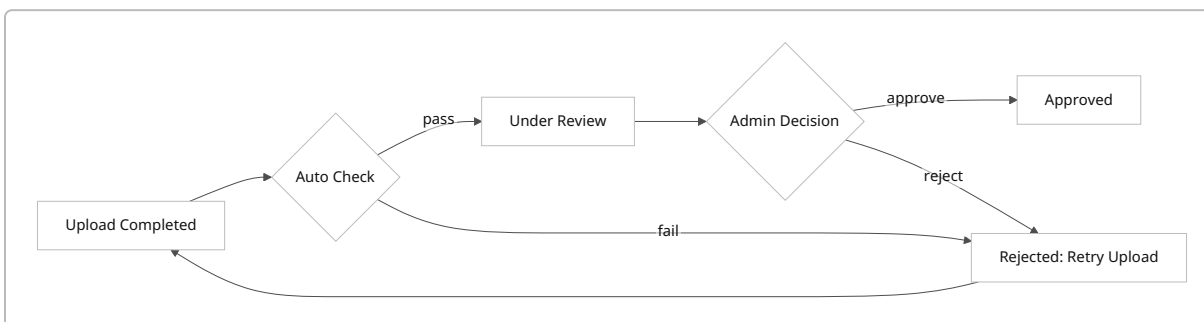
Admins use a protected interface. They query pending KYC requests, view all submitted docs (with OCR text and times), then approve or reject. On action, the system updates the status and notifies the user (via our Notification Service). Audit logs record the admin's decision and any comments.

Document Verification Workflow (States & Transitions)

The **Document Verification** process for each KYC request has these states:

- **Documents Uploaded:** User has provided all requested documents. System has stored them and begun OCR.
- **Under Review:** All OCR checks passed and now either an automated or manual compliance review is in progress.
- **Approved / Rejected:** Final states. Approval means user can invest; rejection means user must fix issues.
- **Re-submission:** If rejected, the user can re-upload corrected documents. The workflow then repeats starting from “Documents Uploaded.”

A simple **flowchart** of this is:



We implement this via a **status field** on the KYC request. Each status change is atomic and triggers notifications. For robustness, if an external API times out (e.g. Texttract delay, UIDAI busy), we can set status to “Retry” or “Pending” and retry the request. All errors (e.g. failed OCR, expired OTP) are caught and returned to the user with instructions, without exposing stack traces.

Technology Stack (Summary)

Layer/Function	Chosen Technology	Rationale
Frontend	React.js (web), React Native (mobile)	Fast UI development, matches Groww’s approach [11†L239-L244] . Supports OAuth flows and image capture.
API Gateway	Node.js (Express) or API Gateway service	Route requests to microservices, rate-limit to prevent brute-force (per zero-retention design [14†L972-L980]).
Auth Service	Node.js (Express/Koa) with Passport/JWT	Stateless auth with JWT. Easy integration.

Layer/Function	Chosen Technology	Rationale
User/Profile Service	Node.js or Spring Boot	CRUD user/profile.
KYC Service	Node.js (Express)	Coordinates multi-step KYC.
Document Service	Node.js (Express) + Multer	Handles file uploads.
OCR/AI Service	Node.js or Python	Calls AWS Textract/Google Vision and Rekognition. Python may simplify AWS SDK usage.
Admin Service	Node.js + React for admin UI	Lightweight UI for compliance officers.
Notification Service	Node.js or Lambda Functions	Publishes SNS topics; uses AWS SES/Twilio for messaging.
Database	MongoDB Atlas (or AWS DocumentDB)	Flexible schema for user and KYC data; horizontal scaling. Primary keys with ObjectId. Use field encryption for PII.
File Storage	AWS S3	Durable, encrypted storage for documents.
OCR API	AWS Textract	Extracts printed and handwritten text, forms, tables [40†L84-L92] .
Face API	AWS Rekognition	Face comparison and liveness detection, fully managed [38†L9-L16] .
Search Index	Elasticsearch	Fast search over KYC requests or audit logs.
Logging	ELK stack or CloudWatch	Centralize service logs. (Comply with eKYC logging rules [8†L82-L90] : do not log raw Aadhaar/DOB.)
Monitoring	Prometheus + Grafana (or AWS CloudWatch)	Collect metrics (latency, success/fail counts) and alerts.
Containerization	Docker	Package each service.
Orchestration	Kubernetes (AWS EKS) or AWS ECS	Auto-scaling, service discovery.
Infrastructure as Code	Terraform or AWS CloudFormation	Define VPC, subnets, EKS cluster, RDS, etc. Ensures reproducibility.
CI/CD	GitHub Actions/Jenkins	Automate build-test-deploy. Deploy to staging then prod.
Secrets Management	AWS Secrets Manager / HashiCorp Vault	Store DB passwords, API keys (DigiLocker key, Aadhaar KUA key) securely.
Encryption	TLS for all transport; AES-256 (KMS) at rest	All data in transit encrypted; sensitive fields (Aadhaar/PAN) encrypted.

Security & Compliance Controls

- **Consent:** Every eKYC call (Aadhaar or DigiLocker) is made only after explicit user consent. We log that consent flag.
- **PII Protection:** We **never store raw Aadhaar numbers or names from KYC responses** in our application DB (only in an encrypted Aadhaar Vault, as required [8†L82-L90]). All logs and DB entries use the UIDAI-issued `uid-token` (pseudonym) instead of the Aadhaar number [8†L82-L90] . PAN numbers are stored as given (they are permitted KYC data).
- **Data Encryption:** Use HTTPS/TLS for all API calls (users to app, app to external services). S3 buckets are encrypted at rest. MongoDB volumes use encryption. AWS KMS manages keys.
- **Least Privilege:** Microservices only have the minimal IAM roles to call needed AWS APIs (e.g. S3, Rekognition). API keys and certificates (DigiLocker, Aadhaar KYC/KUA) are in AWS Secrets Manager.
- **Audit Logs:** Immutable log of all actions (uploads, approvals, declines) with timestamps and actor IDs. Stored securely and only accessible to admins and auditors. We follow UIDAI's log guidelines [8†L82-L90] (no OTPs or raw KYC data in logs).
- **Resilience:** Use HTTPS with strict timeouts. If an external API call fails (e.g. Textract timeout), we retry a couple times. On persistent failures, we mark the KYC request as "retry needed" and prompt the user accordingly.
- **Regulatory Compliance:** For Aadhaar eKYC, we adhere to Aadhaar Act rules: calls to UIDAI go through a KUA interface with certified device, and OTP goes only to the registered mobile. We also comply with SEBI's norm that digital KYC (Aadhaar) is equivalent to in-person verification [35†L98-L102] . (Our frontend flow may require uploading an "IPV consent" form if needed.)

All these measures ensure a secure, zero-retention approach: we handle PII in memory or secure vaults only, trusting certified third-parties to verify identity [14†L931-L939] .

DevOps & Deployment

- **Source Control:** Git with feature branches. Use GitHub/GitLab.
- **CI Pipeline:** On pull request to main branch, run unit tests and static code analysis (lint, security scan). On merge, build Docker images and push to AWS ECR.
- **Staging/Prod:** Deploy containers to separate Kubernetes namespaces (or ECS clusters). Use Helm charts or Terraform modules. Database migrations (if any) run via CI as needed.
- **Infrastructure:** Managed in Terraform: VPC, Subnets, RDS/Mongo cluster, EKS nodes, IAM roles, Load Balancers, etc.
- **Secrets:** Terraform provisions Secrets Manager entries (we only reference them in pods). Pod's env uses IAM roles or Kubernetes secrets synced from Vault.
- **Backups:** Daily automated snapshots of databases and instance stores. S3 has versioning for documents. Test restore procedures periodically.
- **Monitoring:** Prometheus + Grafana (hosted in EKS) scrapes app metrics and cluster health (via kube-state-exporter). Grafana dashboards track onboarding funnel (e.g. how many users at each KYC step). CloudWatch monitors AWS limits. We configure alerts (e.g. PagerDuty, Slack) for job failures or error spikes.
- **Logging:** Container logs go to CloudWatch Logs or ELK, centralized with log level controls. Security events (failed logins, repeated rejections) alert the security team.

- **Scaling:** We set services to auto-scale on CPU/memory/queue depth. For example, scale out KYC Service or OCR workers if Textract calls backlog.
- **Feature Flags & Rollout:** For an internship MVP, feature flags can toggle complex flows (like DigiLocker) if not fully ready.
- **Documentation & Tests:** All APIs are documented (OpenAPI spec). Write unit and integration tests for each service. Have a sandbox mode (simulate DigiLocker and Aadhaar) for development.

Implementation Timeline (Sample 12-week Plan)

- **Weeks 1–2: Setup & Auth** – Define architecture; set up Git/CICD. Implement AuthService (user registration/login with JWT) and mobile OTP. **Deliverables:** Project scaffold, Docker setup, basic React login pages, working JWT login.
- **Weeks 3–4: Profile & KYC Initiation** – Build Profile service and database schema. Create KYC service skeleton. Add "Start KYC" endpoint and UI, with mobile verification integration. **Deliverables:** Profile forms, OTP flow for Aadhaar mobile, KYC start API.
- **Weeks 5–6: PAN/Aadhaar Verification** – Integrate DigiLocker login (sandbox) and/or manual upload. Connect AWS Textract; implement DocumentService. Show extracted fields in UI. **Deliverables:** Working PAN/Aadhaar upload and auto-fill; status tracking.
- **Week 7: Selfie/Liveness** – Integrate Rekognition or equivalent. Build selfie capture page with instructions. Process face-match. **Deliverables:** Selfie verification in the KYC flow.
- **Week 8: Admin Dashboard** – Develop admin UI (React) and APIs to list/review KYC requests. Include approve/reject actions. **Deliverables:** Admin can view pending KYC, inspect docs, approve.
- **Week 9: Notifications & Audit** – Hook up SNS/SES. Send email/SMS at each step (OTP, KYC status). Implement audit log writing for each action. **Deliverables:** Emails sent on KYC approval/rejection; complete audit trail.
- **Week 10: Error Handling & Polish** – Handle edge cases (e.g. duplicate PAN, expired OTP), refine UI (progress bars, help texts). Add re-submission flows. **Deliverables:** Robust error messages, ability to fix issues.
- **Week 11: DevOps & Monitoring** – Finalize Terraform infra. Deploy to a cloud environment (AWS). Set up Prometheus/Grafana dashboards and alerts. **Deliverables:** Live staging environment, basic monitoring up.
- **Week 12: Testing & Documentation** – Full end-to-end tests (including simulated failures). Write final docs (README, API spec). Prepare handover/demos. **Deliverables:** Tested MVP, "How it works" document for new developers.

At each stage, we demo to mentors and iterate based on feedback. This plan is granular enough that a beginner can follow it step-by-step.

References

- Official regulatory guides (UIDAI eKYC specification, SEBI KYC FAQ [\[35†L18-L22\]](#) [\[35†L48-L51\]](#)).
- AWS official docs (Textract Blog [\[40†L84-L92\]](#) , Rekognition Pricing [\[38†L9-L16\]](#)).
- Industry articles (Groww's tech stack [\[11†L239-L244\]](#) , Fenargo guide on digital onboarding [\[22†L1244-L1253\]](#) , SDTimes on zero-retention KYC [\[14†L931-L939\]](#)).
- Fintech best-practices (DigiLocker integration guides, openKYC frameworks).
- Academic surveys on eKYC and liveness detection (for conceptual backing).

These sources shaped our design: using eKYC/Aadhaar for speed, minimizing data retention 【14†L931-L939】 【8†L82-L90】 , and emphasizing user guidance and ML automation 【11†L237-L241】 【40†L84-L92】 . The result is a secure, compliant onboarding system that a new investor can complete quickly and confidently.
